

Causal Survival - RHC Application - IPCW

2026-05-31

Construct Dataset Function (RHC)

```
construct_data_set <- function() {
  Hmisc::getHdata(rhc)
  # primary analysis variables
  time <- pmax(rhc$dthdte, rhc$lstctdte, na.rm = TRUE) - rhc$sadmdte
  Delta <- ifelse(rhc$death == "Yes", 1, 0)
  C <- 1 - Delta
  A <- ifelse(rhc$swang1 == "RHC", 1, 0)

  # demographic and socioeconomic characteristics
  Gender <- rhc$sex
  Age <- as.numeric(rhc$age)
  Race <- droplevels(rhc$race)
  Education <- as.numeric(rhc$edu)
  Income <- rhc$income
  InsuranceClass <- rhc$ninsclas
  ## Patient weight in kilograms
  ## Note: The original dataset encodes some missing values as 0. These values
  ## are retained because replacing them with NA substantially alters the
  ## estimated propensity scores and covariate balance following weighting.
  Weight <- as.numeric(rhc$wtkilo1)

  # pre-existing comorbidities
  hx.vars <- grep("hx$", names(rhc), value = TRUE)
  rhc[hx.vars] <- lapply(rhc[hx.vars], function(x) {
    x <- as.numeric(x)

    factor(
      ifelse(x == 1, "Yes", "No"),
      levels = c("No", "Yes")
    )
  })

  HX.Variables <- rhc[hx.vars]
  names(HX.Variables) <- paste0(
    toupper(sub("hx$", "", names(HX.Variables))),
    "hx"
  )

  # clinical characteristics
  Cancer <- rhc$ca
  Category <- droplevels(rhc$cat1)
  Severity <- as.numeric(rhc$das2d3pc)

  # construct data set
  data.set <- data.frame(
    time = time,
    Delta = Delta,
    C = C,
    A = A,

    Gender = Gender,
```

```
Age = Age,  
Weight = Weight,  
Race = Race,  
Education = Education,  
Income = Income,  
InsuranceClass = InsuranceClass,  
  
HX.Variables,  
  
Cancer = Cancer,  
Category = Category,  
Severity = Severity  
)  
  
return(data.set)  
}
```

Propensity Score Formula Function

```
propensity_score_form <- function(
  data.set, covariates, outcome, additional = ""
) {

  all.covariates <- c(
    covariates,
    additional[additional != ""]
  )

  continuous <- vapply(
    all.covariates,
    function(covariate) {

      x <- data.set[[covariate]]

      if (!is.numeric(x) || is.integer(x)) {
        return(FALSE)
      }

      x <- x[!is.na(x)]

      non.integer.values <- unique(
        x[x != trunc(x)]
      )

      length(non.integer.values) >= nrow(data.set) * 0.1
    },
    logical(1)
  )

  cont.covariates <- all.covariates[continuous]
  non.cont.covariates <- all.covariates[!continuous]

  spline.terms <- paste0(
    "splines::ns(",
    cont.covariates,
    ", df = 2)"
  )

  linear.terms <- non.cont.covariates

  rhs.terms <- c(
    spline.terms,
    linear.terms
  )

  return(as.formula(
    paste(
      outcome,
      "~ 0 +",
      paste(rhs.terms, collapse = " + ")
    )
  ))
}
```

```
)  
))  
}
```

Propensity Score Function

```
propensity_score <- function(  
  data.set,  
  outcome, covariates,  
  propensity.score.form,  
  additional = ""  
) {  
  
  formula <- propensity.score.form(  
    data.set = data.set,  
    covariates = covariates,  
    outcome = outcome,  
    additional = additional  
  )  
  
  # return the fitted propensity score model  
  return(  
    fitted(  
      glm(  
        formula = formula,  
        data = data.set,  
        family = binomial(link = "logit")  
      )  
    )  
  )  
}
```

Plot Propensity Score Function

```
plot_propensity_scores <- function(  
  propensity.score.0, propensity.score.1,  
  cols, fills,  
  title = "Propensity Score Overlap"  
) {  
  
  par(  
    mar = c(2,2,2,0)  
  )  
  
  # plot propensity scores for individuals with A == 0  
  hist(  
    propensity.score.0,  
    col = fills[1],  
    breaks = seq(0,1,by=0.02),  
    ylim = c(0,200),  
    main = title,  
    xlab = "Propensity Scores"  
  )  
  
  # plot propensity scores for individuals with A == 1  
  hist(  
    propensity.score.1,  
    col = fills[2],  
    breaks = seq(0,1,by=0.02),  
    add = T  
  )  
  
  box()  
  
  legend(  
    "topright",  
    legend = c('A = 0', 'A = 1'),  
    fill = cols,  
    border = "black",  
    bty = "n"  
  )  
}
```

Stabilized IPW Function

```
stabilized_IPW <- function(propensity.score, outcome) {  
  
  p.mean <- mean(outcome)  
  
  # calculate stabilized ipws  
  ipw.stabilized <- ifelse(  
    outcome == 1,  
    p.mean / propensity.score,  
    (1 - p.mean) / (1 - propensity.score)  
  )  
  
  return(ipw.stabilized)  
}
```


Plot Multiple IPWs Function

```
plot_ipw <- function(ipws, titles, cols) {

  cols.alpha <- adjustcolor(cols, alpha.f = 0.5)
  par(mar = c(2, 2, 2, 0))

  # common breaks for all IPWs
  breaks <- seq(
    0,
    max(unlist(ipws)) + 0.5,
    by = 0.1
  )

  # plot first ipw distribution
  hist(
    ipws[[1]],
    col = cols.alpha[1],
    breaks = breaks,
    xlim = c(0, 5),
    main = "IPW Comparison",
    ylab = "Frequency",
    xlab = "IPWs"
  )

  # add remaining ipw distributions
  if (length(ipws) > 1) {
    for (i in 2:length(ipws)) {
      hist(
        ipws[[i]],
        col = cols.alpha[i],
        breaks = breaks,
        add = TRUE
      )
    }
  }

  box()

  legend(
    "topright",
    legend = titles,
    fill = cols,
    border = "black",
    bty = "n"
  )
}
```

Weighted Descriptive Statistics: Table Two Function

```
weighted_table_one <- function(data.set, weights, covariates) {  
  # create weighted data set  
  data.set.ipw <- survey::svydesign(  
    ids = ~ 0,  
    data = data.set,  
    weights = weights  
  )  
  
  # create table one for the weighted data set  
  table.one.ipw <- tableone::svyCreateTableOne(  
    vars = covariates,  
    strata = "A",  
    data = data.set.ipw,  
    test = FALSE  
  )  
  
  print(table.one.ipw, smd = TRUE)  
}
```

Plot Covariate Balance

```
plot_cov_balance <- function(  
  data.set, covariates.added, weights.list, weights.titles, cols  
) {  
  
  balance.types <- length(weights.list)  
  
  # access covariate balance before and after weighting  
  balance <- cobalt::bal.tab(  
    as.formula(paste("A ~", covariates.added)),  
    data = data.set,  
    weights = weights.list,  
    method = "weighting",  
    estimand = "ATE",  
    un = TRUE  
  )  
  
  # plot the covariate balance  
  cobalt::love.plot(  
    balance,  
    stats = "mean.diffs",  
    threshold = 0.1,  
    abs = TRUE,  
    line = TRUE,  
    var.order = "unadjusted",  
    sample.names = c("Unadjusted", weights.titles),  
    colors = cols[seq(1, 1+balance.types, 1)],  
    shapes = seq(17, 17-balance.types, -1),  
    limits = c(m = c(0, 0.3)),  
    title = "Covariate Balance Before and After Weighting",  
    labels = TRUE  
  )  
}
```

Estimation

Modified Case-Base Smooth Hazard Model

```
## Modified implementation of fitSmoothHazard() from the casebase package.
## The function extends the standard case-base fitting procedure to allow
## observation-level weights, including inverse probability weights (IPWs).
FSH <- function(
  formula,
  data,
  time,
  weights = rep(1, nrow(data)),
  ratio = 100,
  ...
) {

  # preserve the original function call for reproducibility
  orig.call <- match.call()

  ## Expand "." in a model formula into the complete set of covariates available
  ## in the supplied dataset. This ensures that the modified FSH function can
  ## explicitly evaluate formulas containing "." before fitting the model.
  expand.dot.formula <- function(formula, data = NULL) {

    ## Check whether the formula contains the "." shorthand for all available
    ## predictors
    if (isTRUE("." %in% all.vars(formula))) {

      ## Preserve the original formula attributes while expanding the terms
      formula.attrs <- attributes(formula)

      try.terms <- try(
        terms(formula, data = data),
        silent = TRUE
      )

      ## Replace the shorthand formula with its explicitly expanded version when
      ## the terms object can be constructed successfully
      if (!is(try.terms, "try-error")) {
        formula <- formula(try.terms)
      }

      ## Restore the original formula attributes after expansion
      attributes(formula) <- formula.attrs
    }

    ## Return the expanded formula
    return(formula)
  }

  formula <- expand.dot.formula(formula, data = data)
```

```

# extract the event variable from the left-hand side of the model formula
event.var <- all.vars(formula[[2]])

#extract time variable from the data set
if (missing(time)) {
  var.names <- casebase::checkArgsTimeEvent(
    data = data,
    event = event.var
  )
  time.var <- var.names$time
} else {
  time.var <- time
}

## Identify the distinct event types present in the data to determine whether
## the analysis involves a single event or competing risks
type.events <- sort(unique(data[[event.var]]))

# Case-Base Sampling

## Generate the case-base sample when the input data have not already been
## sampled. The original dataset is retained so that the supplied observation
## weights can be incorporated into the case-base sampling procedure.
if (!inherits(data, "casebaseData")) {

  original.data <- as.data.frame(data)
  original.data$weights <- weights

  ## Sample cases and controls from the person-time data. The sampling ratio
  ## controls the number of person-moments sampled relative to observed cases.
  sample.data <- casebase::sampleCaseBase(
    original.data,
    time.var,
    event.var,
    comprisk = (length(type.events) > 2),
    ratio
  )
} else {
  ## If the input is already a case-base dataset, retain it directly.
  ## The original unsampled data are unavailable at this stage.
  original.data <- data.frame()
  sample.data <- data
}

# Weighted Case-Base Model

## Include the case-base sampling offset in the model formula. This offset
## accounts for the sampling mechanism when estimating the hazard function.
formula <- update(formula, ~ . + offset(offset))

```

```

## Fit a weighted binomial regression model to the case-base sample.
## Observation-level weights allow the model to incorporate IPWs in addition
## to the case-base sampling structure.
fitting.function <- function(formula) glm(
  formula,
  data = sample.data,
  family = binomial,
  weights = weights
)

## Estimate the smooth hazard model using the weighted case-base sample
out <- fitting.function(formula)

# Model Object and Metadata

## Preserve the lower-level glm call while replacing the displayed call with
## the original FSH call for consistency with the custom model interface
out$lower_call <- out$call
out$call <- orig.call

## Store the original data and key variable names for downstream methods
## such as prediction and plotting
out$originalData <- original.data
out$typeEvents <- type.events
out$timeVar <- time.var
out$eventVar <- event.var

## Record the number of case and background person-moments in the sampled
## case-base dataset
num.person.moments <- table(sample.data[[event.var]])
out$num_cm <- num.person.moments[2]
out$num_bm <- num.person.moments[1]
out$formula <- formula

## Store the case-base sampling offset separately, then reset the model's
## offset to zero so that absolute risk estimation can be performed without
## retaining the sampling correction in the prediction stage
out$offset <- out$data$offset
out$data$offset <- 0

## Assign a custom class so that methods defined for single-event case-base
## models can be applied to the fitted object
class(out) <- c("singleEventCB", class(out))

return(out)
}

```

Extract and Report Hazard Ratio

```
## Extract the treatment coefficient, calculate its standard error, and report
## the hazard ratio with its corresponding 95% Wald confidence interval.
extract_print_hr <- function(model, se.func) {

  # Extract the treatment coefficient and its standard error
  beta <- coef(model)["A"]
  se <- sqrt(diag(se.func(model))["A"])

  # Transform the treatment coefficient from the log-hazard ratio scale to the
  # hazard ratio scale and calculate the corresponding 95% Wald confidence
  # interval
  cat("HR-hat:", exp(beta), "\n")
  cat("SE of HR-hat:", se, "\n")
  cat(
    "95% CI: (",
    exp(beta + c(-1, 1) * 1.96 * se),
    ")\n"
  )
}
```

Fit Model

```
fit_model <- function(  
  fitting.function,  
  formula,  
  data.set,  
  seed,  
  extract.print.hr,  
  constant.HR = TRUE,  
  se.func = NULL,  
  ...  
) {  
  ## Set seed to ensure reproducibility of any stochastic procedures used  
  ## during model fitting  
  set.seed(seed)  
  
  ## Evaluate arguments supplied through ...  
  dots <- list(...)  
  
  ## Fit Model  
  model <- do.call(  
    fitting.function,  
    c(  
      list(  
        formula = formula,  
        data = data.set  
      ),  
      dots  
    )  
  )  
  
  if (constant.HR) {  
    ## Extract and report the hazard ratio  
    extract.print.hr(model, se.func)  
  }  
  
  return(model)  
}
```


casebase Variable HR Extraction

```
casebase_var_hr <- function(model, time.grid) {  
  invisible(pdf(NULL))  
  
  out <- plot(  
    model,  
    type = "hr",  
    newdata = data.frame(  
      time = time.grid,  
      A = 0  
    ),  
    var = "A",  
    xvar = "time",  
    ci = TRUE  
  )  
  
  invisible(dev.off())  
  
  return(out)  
}
```

Plot time-dependent Hazard Ratios

```
plot_time_dep_hr <- function(
  models.list, main.title, time.grid,
  casebase.var.hr,
  titles = "", cols = "red"
) {

  # global function consts
  hr.list <- list()
  cols.alpha <- adjustcolor(cols, alpha.f = 0.2)

  # extract all hr curves
  for (i in 1:length(models.list)) {
    hr.list[[i]] <- casebase.var.hr(
      model = models.list[[i]],
      time.grid = time.grid
    )
  }

  # common time grid
  common.time <- time.grid

  # common HR grid
  common.hr <- range(
    unlist(lapply(hr.list, function(x) x$hazard_ratio)),
    na.rm = TRUE
  )

  # define common time and hr grids here then use in plot below

  # define base plot
  plot(
    NA,
    xlim = range(common.time),
    ylim = range(common.hr),
    type = "n",
    xlab = "Time",
    ylab = "Hazard Ratio",
    main = main.title
  )

  # plot all ci bands
  for (i in 1:length(hr.list)) {
    polygon(
      c(hr.list[[i]]$time, rev(hr.list[[i]]$time)),
      c(hr.list[[i]]$lowerbound, rev(hr.list[[i]]$upperbound)),
      col = cols.alpha[i],
      border = NA
    )
  }

  # plot all hr curves on top
```

```

for (i in 1:length(hr.list)) {
  lines(
    hr.list[[i]]$time,
    hr.list[[i]]$hazard_ratio,
    col = cols[i],
    lwd = 2
  )
}

# plot null hr reference line
abline(
  h = 1,
  col = "black",
  lty = 2
)

if (length(models.list) > 1) {
  # create legend
  legend(
    "bottomleft",
    legend = titles,
    col = cols,
    lwd = 2,
    bty = "n"
  )
}
}

```

Restricted Mean Survival Time (RMST)

```
## Numerically integrate a survival curve over the specified follow-up period  
## using the trapezoidal rule. The cumulative integral at each time point  
## represents the RMST up to that point.  
rmst_calculation <- function(time, surv) {  
  time.diff <- diff(time)  
  midpoints <- (head(surv, -1) + tail(surv, -1)) / 2  
  c(0, cumsum(midpoints * time.diff))  
}
```

MS-Cox RMST and Standard Error

```
## Calculate the Counter-factual RMST contrast and its standard error from the
## marginal structural Cox model. The RMST contrast is defined as the
## difference in restricted mean survival between RHC and no RHC.
cox_rmst_list <- function(
  ms.cox, time.grid, cox.psi.se, survival.curves.data, rmst.calculation
) {

  # Counter-factual Survival Curves

  ## Estimate the Counter-factual survival curves under both treatment levels
  ## from the fitted marginal structural Cox model
  surv <- survival::survfit(
    ms.cox,
    newdata = survival.curves.data
  )

  ## Interpolate the Counter-factual survival curve under no RHC onto the
  ## specified time grid
  S0 <- approx(
    surv$time,
    surv$surv[, 1],
    time.grid,
    method = "constant",
    rule = 2
  )$y

  ## Interpolate the Counter-factual survival curve under RHC onto the
  ## specified time grid
  S1 <- approx(
    surv$time,
    surv$surv[, 2],
    time.grid,
    method = "constant",
    rule = 2
  )$y

  # RMST Contrast

  ## Calculate the RMST contrast as the difference between the restricted mean
  ## survival under RHC and no RHC
  rmst <- rmst.calculation(time.grid, S1) - rmst.calculation(time.grid, S0)

  # Delta-Method Standard Error

  ## Construct the gradient of the RMST functional with respect to the
  ## treatment log-hazard coefficient. For the Cox model, the derivative
  ## involves the integral of  $S_1(t) \log\{S_1(t)\}$  over the restriction period.
  integrand <- ifelse(S1 > 0, S1 * log(S1), 0)
  cox.gradient <- rmst.calculation(time.grid, integrand)

  ## Propagate uncertainty from the estimated Cox treatment coefficient to the
```

```
## RMST contrast using the delta method
rmst.se <- abs(cox.gradient) * cox.psi.se

return(list(
  rmst = rmst,
  rmst.se = rmst.se
))
}
```

casebase Counter-factual Survival Function

```
## Construct the Counter-factual survival function under treatment level a
## directly from the case-base model's design matrix
casebase_surv <- function(beta, a, time.grid, term.form) {

  ## Create prediction data over the specified time grid under treatment
  ## level a
  new.data <- data.frame(
    time = time.grid,
    A = a,
    offset = 0
  )

  ## Construct the model matrix using the same terms as the fitted model and
  ## align its columns with the estimated coefficient vector
  covariates <- model.matrix(term.form, data = new.data)
  covariates <- covariates[, names(beta), drop = FALSE]

  ## Convert the linear predictor to the estimated hazard function
  lambda <- exp(as.vector(covariates %*% beta))

  ## Numerically integrate the estimated hazard using the trapezoidal rule
  ## to obtain the cumulative hazard
  Lambda <- c(
    0,
    cumsum(
      diff(time.grid) * (head(lambda, -1) + tail(lambda, -1)) / 2
    )
  )

  ## Convert cumulative hazard to survival probability
  return(exp(-Lambda))
}
```

RMST Contrast

```
## Define the RMST contrast as a function of the model coefficient vector.
## This function is subsequently used to obtain the numerical gradient
## required for the delta-method standard error.
rmst_diff <- function(
  beta, time.grid, casebase.surv, rmst.calculation, term.form
) {

  rmst.A1 <- rmst.calculation(
    time = time.grid,
    surv = casebase.surv(
      beta = beta,
      a = 1,
      time.grid = time.grid,
      term.form = term.form
    )
  )

  rmst.A0 <- rmst.calculation(
    time = time.grid,
    surv = casebase.surv(
      beta = beta,
      a = 0,
      time.grid = time.grid,
      term.form = term.form
    )
  )

  return(rmst.A1 - rmst.A0)
}
```


IPW Case-Base RMST and Standard Error

```
## Calculate the RMST contrast and its standard error from the IPW case-base
## model. The survival curves are reconstructed from the fitted model and the
## variance of the RMST contrast is obtained using a numerical delta method.
casebase_rmst_list <- function(
  casebase.ipw, casebase.var, time.grid, constant.HR,
  casebase.surv, rmst.diff, rmst.calculation
) {

  ## Extract the estimated model coefficients and the corresponding model
  ## terms used to construct the design matrix
  beta.hat <- coef(casebase.ipw)
  term.form <- delete.response(terms(casebase.ipw))

  ## Evaluate the RMST contrast at the estimated model coefficients
  rmst <- rmst.diff(
    beta = beta.hat,
    time.grid = time.grid,
    casebase.surv = casebase.surv,
    rmst.calculation = rmst.calculation,
    term.form = term.form
  )

  # Numerical Gradient

  ## Calculate the numerical gradient of the RMST contrast with respect to
  ## each model coefficient using a central finite-difference approximation
  num.coefs <- length(beta.hat)
  num.time.points <- length(time.grid)

  gradient <- matrix(NA_real_, nrow = num.time.points, ncol = num.coefs)
  colnames(gradient) <- names(beta.hat)

  ## Step size used for the finite-difference approximation
  step.size <- 1e-6

  ## Perturb each coefficient in turn and approximate the corresponding
  ## derivative of the RMST contrast
  for (j in seq_len(num.coefs)) {
    beta.plus <- beta.minus <- beta.hat
    beta.plus[j] <- beta.plus[j] + step.size
    beta.minus[j] <- beta.minus[j] - step.size

    rmst.beta.plus <- rmst.diff(
      beta = beta.plus,
      time.grid = time.grid,
      casebase.surv = casebase.surv,
      rmst.calculation = rmst.calculation,
      term.form = term.form
    )

    rmst.beta.minus <- rmst.diff(
```

```

    beta = beta.minus,
    time.grid = time.grid,
    casebase.surv = casebase.surv,
    rmst.calculation = rmst.calculation,
    term.form = term.form
  )

  gradient[, j] <- (rmst.beta.plus - rmst.beta.minus) / (2 * step.size)
}

# Delta-Method Standard Error

## Align the model covariance matrix with the coefficient ordering used by
## the numerical gradient before applying the delta-method variance formula
casebase.var <- casebase.var[colnames(gradient), colnames(gradient), drop = FALSE]

## Calculate the RMST standard error at each restriction time using
## Var{g(beta)} = gradient' Var(beta) gradient
rmst.se <- vapply(
  seq_len(num.time.points),
  function(i) {
    grad.row <- gradient[i, ]
    sqrt(drop(t(grad.row) %*% casebase.var %*% grad.row))
  },
  numeric(1)
)

return(list(
  rmst = rmst,
  rmst.se = rmst.se
))
}

```

Counter-factual Survival Curves: MS-Cox and IPW Case-Base Models

Calculate Counter-factual Survival Curves Function (MS-Cox)

```
cox_csc <- function(model, new.data, time.grid) {  
  surv <- survival::survfit(  
    model,  
    newdata = new.data  
  )  
  
  ## Interpolate the estimated survival curve under no RHC onto the common  
  ## time grid used for subsequent comparison and RMST calculation  
  S0 <- approx(  
    surv$time,  
    surv$surv[, 1],  
    xout = time.grid,  
    method = "constant",  
    rule = 2  
  )$y  
  
  ## Interpolate the corresponding lower 95% confidence limit under no RHC  
  S0.lower <- approx(  
    surv$time,  
    surv$lower[, 1],  
    xout = time.grid,  
    method = "constant",  
    rule = 2  
  )$y  
  
  ## Interpolate the corresponding upper 95% confidence limit under no RHC  
  S0.upper <- approx(  
    surv$time,  
    surv$upper[, 1],  
    xout = time.grid,  
    method = "constant",  
    rule = 2  
  )$y  
  
  ## Interpolate the estimated survival curve under RHC onto the common time grid  
  S1 <- approx(  
    surv$time,  
    surv$surv[, 2],  
    xout = time.grid,  
    method = "constant",  
    rule = 2  
  )$y  
  
  ## Interpolate the corresponding lower 95% confidence limit under RHC  
  S1.lower <- approx(  
    surv$time,
```

```

    surv$lower[, 2],
    xout = time.grid,
    method = "constant",
    rule = 2
  )$y

  ## Interpolate the corresponding upper 95% confidence limit under RHC
  S1.upper <- approx(
    surv$time,
    surv$upper[, 2],
    xout = time.grid,
    method = "constant",
    rule = 2
  )$y

  return(list(
    S0 = S0,
    S0.lower = S0.lower, S0.upper = S0.upper,
    S1 = S1,
    S1.lower = S1.lower, S1.upper = S1.upper
  ))
}

```

Calculate Counter-factual Survival Curves Function (Case Base)

```
casebase_csc <- function(
  seed, data.set, model, weights, new.data, time.grid, FSH
) {
  surv <- casebase::absoluteRisk(
    object = model,
    time = time.grid,
    newdata = new.data,
    type = "survival"
  )

  ## Extract the estimated survival curves for no RHC and RHC, respectively
  S0 <- surv[, 2]
  S1 <- surv[, 3]

  # Bootstrap Confidence Intervals for the IPW Case-Base Curves

  ## Use nonparametric bootstrap resampling to obtain pointwise confidence
  ## intervals for the Counter-factual survival curves estimated by the IPW
  ## case-base model
  set.seed(seed)
  num.samples <- 200

  ## For each bootstrap sample, refit the IPW case-base model and obtain
  ## Counter-factual survival estimates under both treatment levels
  boot.surv <- replicate(num.samples, {
    boot.idx <- sample(nrow(data.set), replace = TRUE)
    boot.data <- data.set[boot.idx, ]

    boot.fit <- FSH(
      Delta ~ survival::pspline(time, df = 2) * A,
      data = boot.data,
      ratio = 10,
      weights = weights[boot.idx]
    )

    boot.surv <- casebase::absoluteRisk(
      boot.fit,
      time = time.grid,
      newdata = new.data,
      type = "survival"
    )

    list(
      S0 = boot.surv[, 2],
      S1 = boot.surv[, 3]
    )
  })

  ## Organize the bootstrap estimates into separate matrices for the no-RHC
  ## and RHC Counter-factual survival curves
  S0.boot <- apply(1:num.samples, function(i) boot.surv[["S0", i]])
```

```

S1.boot <- sapply(1:num.samples, function(i) boot.surv[["S1", i]])

## Calculate the pointwise lower 95% bootstrap confidence limit for the
## no-RHC Counter-factual survival curve
S0.lower <- apply(
  S0.boot,
  1,
  quantile,
  probs = 0.025,
  na.rm = TRUE
)

## Calculate the pointwise upper 95% bootstrap confidence limit for the
## no-RHC Counter-factual survival curve
S0.upper <- apply(
  S0.boot,
  1,
  quantile,
  probs = 0.975,
  na.rm = TRUE
)

## Calculate the pointwise lower 95% bootstrap confidence limit for the
## RHC Counter-factual survival curve
S1.lower <- apply(
  S1.boot,
  1,
  quantile,
  probs = 0.025,
  na.rm = TRUE
)

## Calculate the pointwise upper 95% bootstrap confidence limit for the
## RHC Counter-factual survival curve
S1.upper <- apply(
  S1.boot,
  1,
  quantile,
  probs = 0.975,
  na.rm = TRUE
)

return(list(
  S0 = S0,
  S0.lower = S0.lower, S0.upper = S0.upper,
  S1 = S1,
  S1.lower = S1.lower, S1.upper = S1.upper
))
}

```

Plot Counter-factual Survival Curves

```
plot_csc <- function(
  time.grid,
  S0, S0.lower, S0.upper,
  S1, S1.lower, S1.upper,
  main.title,
  fills, cols
) {

  plot(
    NA,
    xlim = range(time.grid),
    ylim = c(0, 1),
    type = "n",
    xlab = "Time",
    ylab = "Survival Probability",
    main = main.title
  )

  ## Add the 95% confidence bands
  polygon(
    c(time.grid, rev(time.grid)),
    c(S0.lower, rev(S0.upper)),
    col = fills[1],
    border = NA
  )

  polygon(
    c(time.grid, rev(time.grid)),
    c(S1.lower, rev(S1.upper)),
    col = fills[2],
    border = NA
  )

  ## Overlay the estimated survival curves. The MS-Cox curve is a step function,
  ## whereas the case-base model produces a smooth survival curve.
  lines(time.grid, S0, type = "s", lwd = 2, col = cols[1], lty = 1)
  lines(time.grid, S1, type = "s", lwd = 2, col = cols[2], lty = 1)

  ## Identify the two estimation approaches in the comparison plot
  legend(
    "topright",
    legend = c("No RHC", "RHC"),
    col = cols[1:2],
    lty = c(1, 1),
    lwd = 2,
    bty = "n"
  )
}
```

RMST Comparison: MS-Cox vs. IPW Case-Base

```
rmst_stats <- function(
  models.list, time.grid, new.data,
  cox.rmst.list, casebase.rmst.list,
  rmst.calculation, rmst.diff, casebase.surv
) {
  cox.psi.se <- sqrt(diag(vcov(models.list$cox))["A"])
  casebase.var <- sandwich::vcovHC(models.list$casebase, type = "HCO")

  cox.rmst.list <- cox.rmst.list(
    ms.cox = models.list$cox,
    time.grid = time.grid,
    cox.psi.se = cox.psi.se,
    survival.curves.data = new.data,
    rmst.calculation = rmst.calculation
  )
  casebase.rmst.list <- casebase.rmst.list(
    casebase.ipw = models.list$casebase,
    casebase.var = casebase.var,
    time.grid = time.grid,
    constant.HR = TRUE,
    casebase.surv = casebase.surv,
    rmst.diff = rmst.diff,
    rmst.calculation = rmst.calculation
  )

  cox.lower <- cox.rmst.list$rmst - 1.96 * cox.rmst.list$rmst.se
  cox.upper <- cox.rmst.list$rmst + 1.96 * cox.rmst.list$rmst.se

  casebase.lower <- casebase.rmst.list$rmst - 1.96 * casebase.rmst.list$rmst.se
  casebase.upper <- casebase.rmst.list$rmst + 1.96 * casebase.rmst.list$rmst.se

  return(list(
    time.grid = time.grid,
    cox.rmst = cox.rmst.list$rmst,
    cox.rmst.se = cox.rmst.list$rmst.se,
    cox.lower = cox.lower,
    cox.upper = cox.upper,
    casebase.rmst = casebase.rmst.list$rmst,
    casebase.rmst.se = casebase.rmst.list$rmst.se,
    casebase.lower = casebase.lower,
    casebase.upper = casebase.upper
  ))
}
```


Plot RMST Comparison

```
plot_rmst_comparison <- function(
  rmst.stats,
  fills, cols,
  additional.title = "") {
  ylim <- range(
    c(
      rmst.stats$cox.lower, rmst.stats$cox.upper,
      rmst.stats$casebase.lower, rmst.stats$casebase.upper
    ),
    na.rm = TRUE
  )

  plot(
    NA,
    xlim = range(rmst.stats$time.grid),
    ylim = ylim,
    type = "n",
    xlab = expression(tau),
    ylab = "RMST Difference",
    main = paste("Causal RMST up to Tau", additional.title)
  )

  # 95% Confidence Bands

  ## Add the pointwise 95% confidence band for the MS-Cox RMST contrast
  polygon(
    c(rmst.stats$time.grid, rev(rmst.stats$time.grid)),
    c(rmst.stats$cox.lower, rev(rmst.stats$cox.upper)),
    col = fills[1],
    border = NA
  )

  ## Add the pointwise 95% confidence band for the IPW case-base RMST contrast
  polygon(
    c(rmst.stats$time.grid, rev(rmst.stats$time.grid)),
    c(rmst.stats$casebase.lower, rev(rmst.stats$casebase.upper)),
    col = fills[2],
    border = NA
  )

  # RMST Contrast Estimates

  ## Overlay the estimated causal RMST contrast from the MS-Cox model
  lines(rmst.stats$time.grid, rmst.stats$cox.rmst, col = cols[1], lwd = 2)

  ## Overlay the estimated causal RMST contrast from the IPW case-base model
  lines(rmst.stats$time.grid, rmst.stats$casebase.rmst, col = cols[2], lwd = 2)

  ## Add a reference line at an RMST difference of zero, corresponding to equal
## restricted mean survival under the two treatment conditions
  abline(h = 0, lty = 3)
```

```
# Model Legend

## Identify the two estimation approaches in the comparison plot
legend(
  "bottomleft",
  legend = c("MS-Cox", "IPW Case-Base"),
  col = cols[1:2],
  lwd = 2,
  bty = "n"
)
}
```

Data Preparation

```
## Set random seed for output reproducibility  
seed = 123
```

Construct Analysis Dataset

```
data_set <- construct_data_set()  
  
covariates_vector <- names(data_set)[-1:4]  
covariates_added <- paste(covariates_vector, collapse = " + ")
```

Plotting Parameters

```
# two main colors used in different plots  
colors <- c("black", "red", "blue", "green")  
  
# lighter shades of colors for confidence bands and overlapped plots  
fills <- adjustcolor(colors, alpha.f = 0.2)  
dark_fills <- adjustcolor(colors, alpha.f = 0.5)
```

Calculate IPTW, IPCW and Combined Weights

```
# calculate treatment propensity scores
treatment_ps <- propensity_score(
  data.set = data_set,
  outcome = "A",
  covariates = covariates_vector,
  propensity.score.form = propensity_score_form
)
treatment_ps_0 <- treatment_ps[data_set$A == 0]
treatment_ps_1 <- treatment_ps[data_set$A == 1]

# calculate censoring propensity scores
censoring_ps <- propensity_score(
  data.set = data_set,
  outcome = "C",
  covariates = covariates_vector,
  propensity.score.form = propensity_score_form,
  additional = "A"
)
censoring_ps_0 <- censoring_ps[data_set$A == 0]
censoring_ps_1 <- censoring_ps[data_set$A == 1]

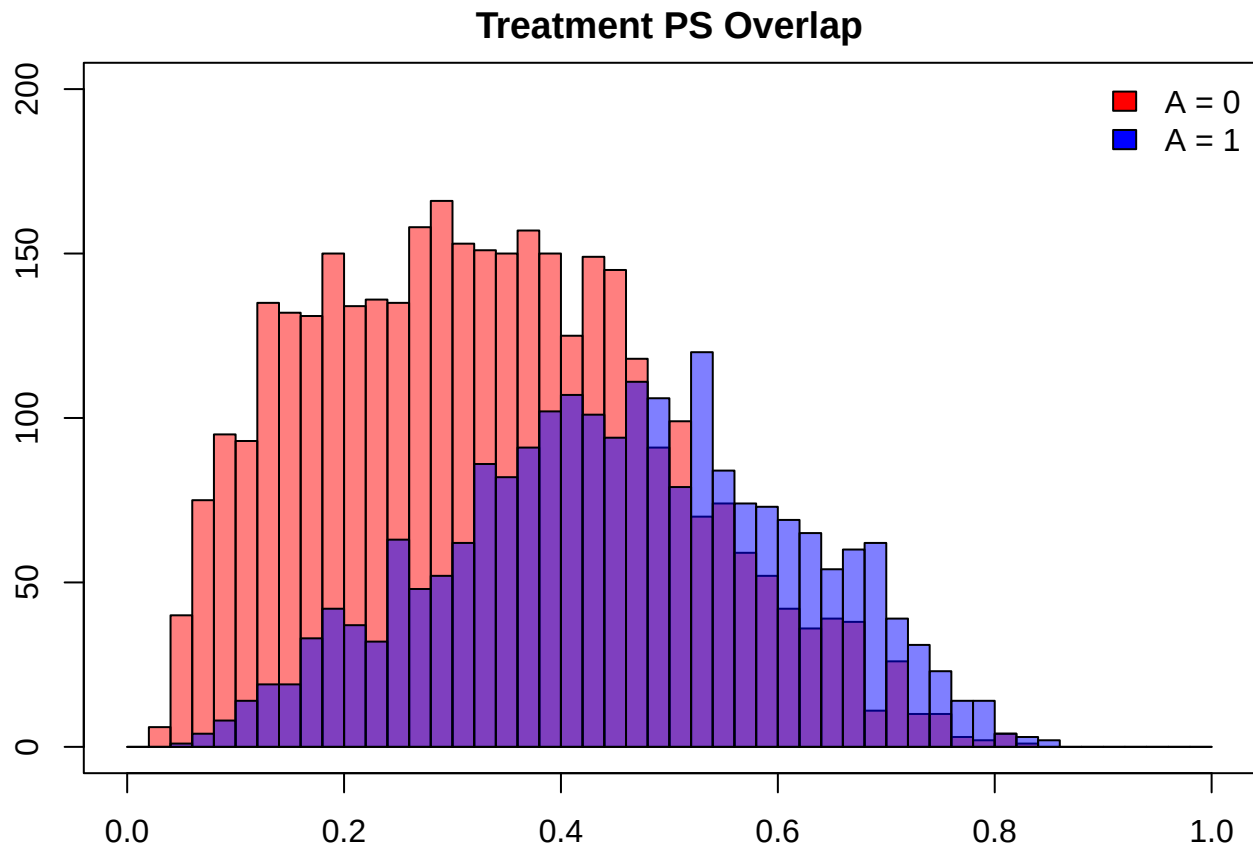
# calculate iptw and ipcw
iptw <- stabilized_IPW(
  propensity.score = treatment_ps,
  outcome = data_set$A
)

ipcw <- stabilized_IPW(
  propensity.score = censoring_ps,
  outcome = data_set$C
)

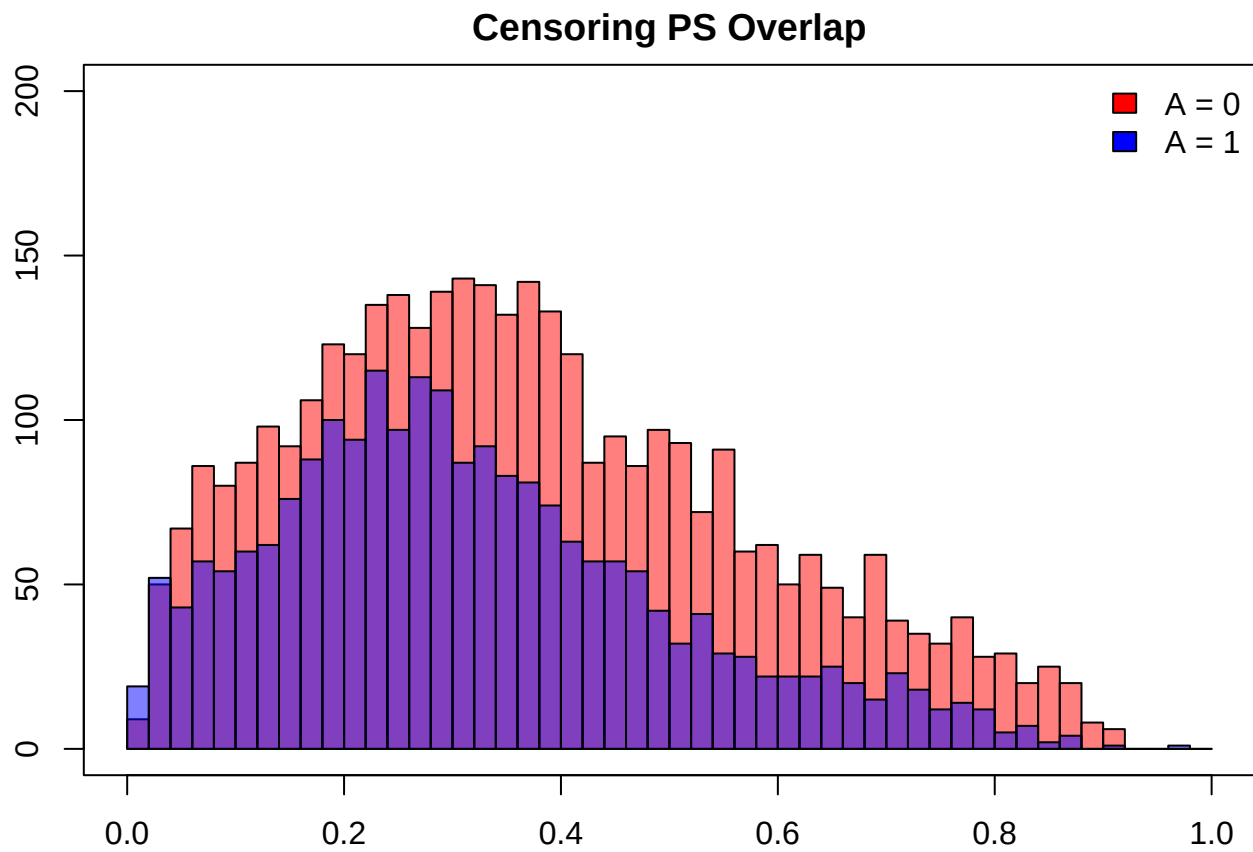
# calculate combined weights
combined_weights <- iptw * ipcw
```

Plot Propensity Scores

```
plot_propensity_scores(  
  propensity.score.0 = treatment_ps_0, propensity.score.1 = treatment_ps_1,  
  cols = colors[2:3], fills = dark_fills[2:3],  
  title = "Treatment PS Overlap"  
)
```

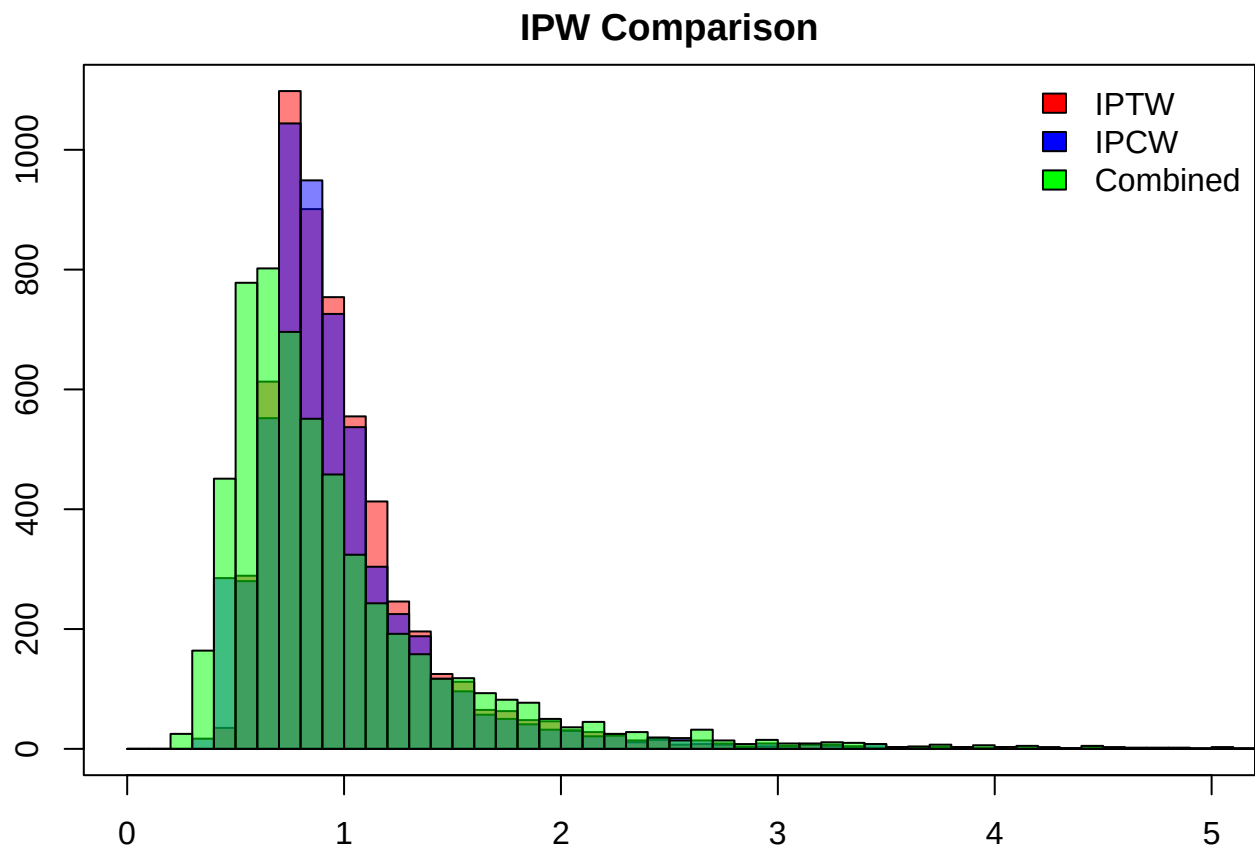


```
plot_propensity_scores(  
  propensity.score.0 = censoring_ps_0, propensity.score.1 = censoring_ps_1,  
  cols = colors[2:3], fills = dark_fills[2:3],  
  title = "Censoring PS Overlap"  
)
```



Plot IPW Distributions

```
plot_ipw(  
  ipws = list(iptw, ipcw, combined_weights),  
  titles = c("IPTW", "IPCW", "Combined"),  
  cols = colors[2:4]  
)
```



Descriptive Statistics: Table One

```
# create table one for the unweighted original data set
table_one_original <- tableone::CreateTableOne(
  vars = covariates_vector,
  strata = "A",
  data = data_set,
  test = FALSE
)

print(table_one_original, smd = TRUE)
```

##	Stratified by A		
##	0	1	SMD
## n	3551	2184	
## Gender = Male (%)	1914 (53.9)	1278 (58.5)	0.093
## Age (mean (SD))	61.76 (17.29)	60.75 (15.63)	0.061
## Weight (mean (SD))	65.04 (29.50)	72.36 (27.73)	0.256
## Race (%)			0.036
## white	2753 (77.5)	1707 (78.2)	
## black	585 (16.5)	335 (15.3)	
## other	213 (6.0)	142 (6.5)	
## Education (mean (SD))	11.57 (3.13)	11.86 (3.16)	0.091
## Income (%)			0.142
## Under \$11k	2081 (58.6)	1145 (52.4)	
## \$11-\$25k	713 (20.1)	452 (20.7)	
## \$25-\$50k	500 (14.1)	393 (18.0)	
## > \$50k	257 (7.2)	194 (8.9)	
## InsuranceClass (%)			0.194
## No insurance	186 (5.2)	136 (6.2)	
## Private	967 (27.2)	731 (33.5)	
## Medicare	947 (26.7)	511 (23.4)	
## Medicaid	454 (12.8)	193 (8.8)	
## Private & Medicare	746 (21.0)	490 (22.4)	
## Medicare & Medicaid	251 (7.1)	123 (5.6)	
## CARDIOhx = Yes (%)	567 (16.0)	446 (20.4)	0.116
## CHFhx = Yes (%)	596 (16.8)	425 (19.5)	0.070
## DEMENThx = Yes (%)	413 (11.6)	151 (6.9)	0.163
## PSYCHhx = Yes (%)	286 (8.1)	100 (4.6)	0.143
## CHRPUHx = Yes (%)	774 (21.8)	315 (14.4)	0.192
## RENALhx = Yes (%)	149 (4.2)	106 (4.9)	0.032
## LIVERhx = Yes (%)	265 (7.5)	136 (6.2)	0.049
## GIBLEDhx = Yes (%)	131 (3.7)	54 (2.5)	0.070
## MALIGHx = Yes (%)	872 (24.6)	444 (20.3)	0.101
## IMMUNhx = Yes (%)	907 (25.5)	636 (29.1)	0.080
## TRANSHx = Yes (%)	335 (9.4)	327 (15.0)	0.170
## AMIhx = Yes (%)	105 (3.0)	95 (4.3)	0.074
## Cancer (%)			0.107
## No	2652 (74.7)	1727 (79.1)	
## Yes	638 (18.0)	334 (15.3)	
## Metastatic	261 (7.4)	123 (5.6)	
## Category (%)			0.583

##	ARF	1581 (44.5)	909 (41.6)	
##	COPD	399 (11.2)	58 (2.7)	
##	CHF	247 (7.0)	209 (9.6)	
##	Cirrhosis	175 (4.9)	49 (2.2)	
##	Coma	341 (9.6)	95 (4.3)	
##	Colon Cancer	6 (0.2)	1 (0.0)	
##	Lung Cancer	34 (1.0)	5 (0.2)	
##	MOSF w/Malignancy	241 (6.8)	158 (7.2)	
##	MOSF w/Sepsis	527 (14.8)	700 (32.1)	
##	Severity (mean (SD))	20.37 (5.48)	20.70 (5.03)	0.063

Weighted Table One (Intention-To-Treat)

```
weighted_table_one(
  data.set = data_set,
  weights = iptw,
  covariates = covariates_vector
)
```

##	Stratified by A			
##	0	1		SMD
## n	3555.7	2172.2		
## Gender = Male (%)	1971.8 (55.5)	1207.5 (55.6)		0.003
## Age (mean (SD))	61.31 (16.66)	61.33 (16.70)		0.001
## Weight (mean (SD))	67.88 (28.93)	67.96 (28.84)		0.003
## Race (%)				0.005
## white	2760.0 (77.6)	1689.8 (77.8)		
## black	577.5 (16.2)	351.2 (16.2)		
## other	218.2 (6.1)	131.2 (6.0)		
## Education (mean (SD))	11.69 (3.13)	11.70 (3.08)		0.004
## Income (%)				0.005
## Under \$11k	1997.0 (56.2)	1220.1 (56.2)		
## \$11-\$25k	725.0 (20.4)	439.7 (20.2)		
## \$25-\$50k	555.4 (15.6)	340.5 (15.7)		
## > \$50k	278.4 (7.8)	172.0 (7.9)		
## InsuranceClass (%)				0.022
## No insurance	198.8 (5.6)	121.0 (5.6)		
## Private	1046.8 (29.4)	635.5 (29.3)		
## Medicare	895.8 (25.2)	554.5 (25.5)		
## Medicaid	404.3 (11.4)	255.5 (11.8)		
## Private & Medicare	780.2 (21.9)	474.3 (21.8)		
## Medicare & Medicaid	229.9 (6.5)	131.4 (6.0)		
## CARDIOhx = Yes (%)	637.5 (17.9)	403.8 (18.6)		0.017
## CHFhx = Yes (%)	635.0 (17.9)	393.9 (18.1)		0.007
## DEMENThx = Yes (%)	347.8 (9.8)	206.9 (9.5)		0.009
## PSYCHhx = Yes (%)	235.8 (6.6)	136.5 (6.3)		0.014
## CHRPUhx = Yes (%)	667.9 (18.8)	405.4 (18.7)		0.003
## RENALhx = Yes (%)	163.7 (4.6)	105.0 (4.8)		0.011
## LIVERhx = Yes (%)	250.7 (7.0)	151.6 (7.0)		0.003
## GIBLEDhx = Yes (%)	114.3 (3.2)	66.6 (3.1)		0.008
## MALIGHx = Yes (%)	816.5 (23.0)	507.1 (23.3)		0.009
## IMMUNhx = Yes (%)	964.5 (27.1)	597.0 (27.5)		0.008
## TRANShx = Yes (%)	412.9 (11.6)	257.1 (11.8)		0.007
## AMIhx = Yes (%)	119.5 (3.4)	74.9 (3.4)		0.005
## Cancer (%)				0.011
## No	2714.1 (76.3)	1649.6 (75.9)		
## Yes	603.0 (17.0)	377.1 (17.4)		
## Metastatic	238.6 (6.7)	145.6 (6.7)		
## Category (%)				0.012
## ARF	1542.2 (43.4)	952.4 (43.8)		
## COPD	282.8 (8.0)	167.5 (7.7)		
## CHF	281.0 (7.9)	172.2 (7.9)		
## Cirrhosis	138.8 (3.9)	84.2 (3.9)		

##	Coma	270.3 (7.6)	163.0 (7.5)	
##	Colon Cancer	4.4 (0.1)	2.8 (0.1)	
##	Lung Cancer	24.1 (0.7)	14.6 (0.7)	
##	MOSF w/Malignancy	246.0 (6.9)	150.3 (6.9)	
##	MOSF w/Sepsis	766.4 (21.6)	465.3 (21.4)	
##	Severity (mean (SD))	20.50 (5.32)	20.53 (5.25)	0.006

Weighted Table One (Per-Protocol)

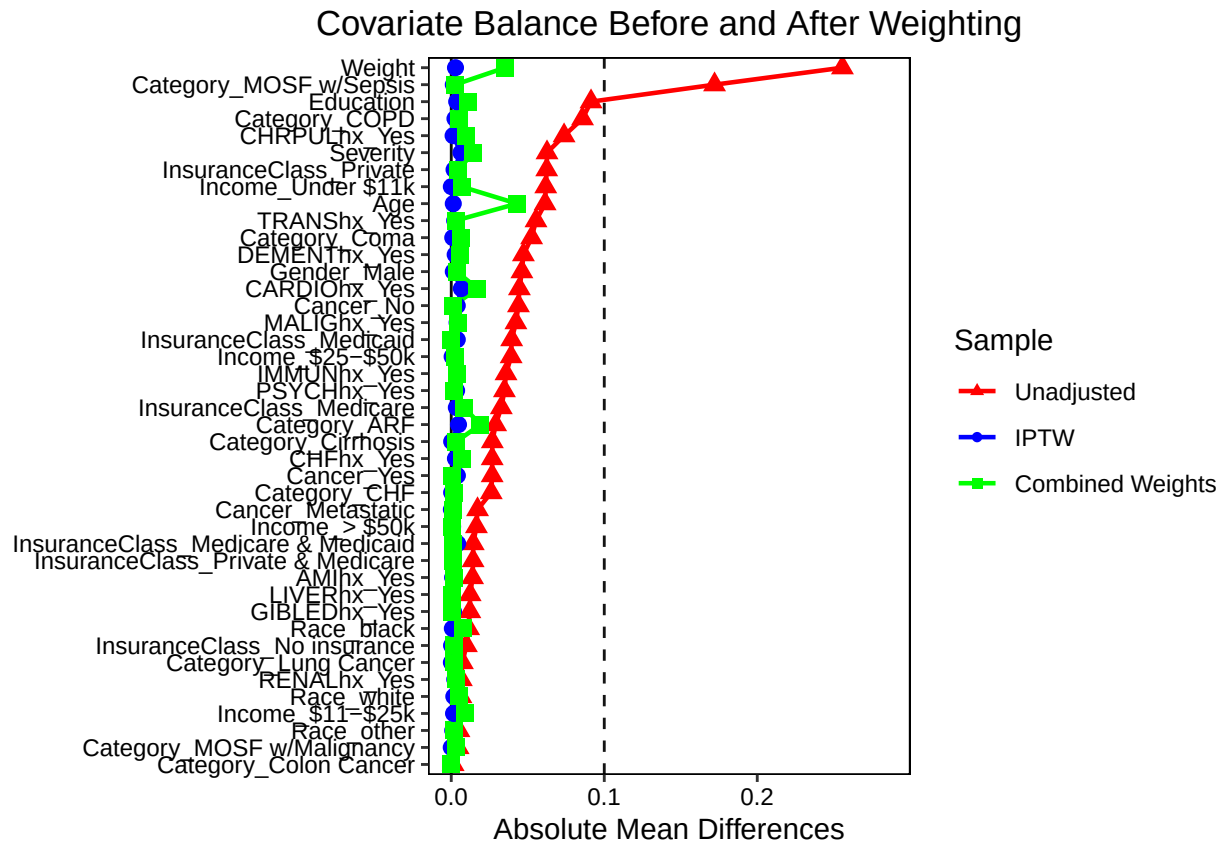
```
weighted_table_one(
  data.set = data_set,
  weights = combined_weights,
  covariates = covariates_vector
)
```

##	Stratified by A			
##	0	1		SMD
## n	3586.0	2138.4		
## Gender = Male (%)	1968.0 (54.9)	1181.2 (55.2)		0.007
## Age (mean (SD))	61.70 (16.55)	60.99 (16.69)		0.043
## Weight (mean (SD))	68.08 (29.13)	69.09 (29.44)		0.034
## Race (%)				0.021
## white	2776.3 (77.4)	1666.9 (78.0)		
## black	596.9 (16.6)	340.1 (15.9)		
## other	212.8 (5.9)	131.3 (6.1)		
## Education (mean (SD))	11.71 (3.19)	11.74 (3.12)		0.011
## Income (%)				0.022
## Under \$11k	2001.3 (55.8)	1179.1 (55.1)		
## \$11-\$25k	724.7 (20.2)	450.9 (21.1)		
## \$25-\$50k	576.7 (16.1)	339.0 (15.9)		
## > \$50k	283.4 (7.9)	169.4 (7.9)		
## InsuranceClass (%)				0.021
## No insurance	182.9 (5.1)	112.3 (5.3)		
## Private	1056.2 (29.5)	639.9 (29.9)		
## Medicare	911.0 (25.4)	525.2 (24.6)		
## Medicaid	413.8 (11.5)	247.0 (11.6)		
## Private & Medicare	794.9 (22.2)	475.8 (22.3)		
## Medicare & Medicaid	227.2 (6.3)	138.1 (6.5)		
## CARDIOhx = Yes (%)	638.6 (17.8)	417.4 (19.5)		0.044
## CHFhx = Yes (%)	667.8 (18.6)	382.7 (17.9)		0.019
## DEMENThx = Yes (%)	354.1 (9.9)	199.3 (9.3)		0.019
## PSYCHhx = Yes (%)	222.9 (6.2)	128.5 (6.0)		0.009
## CHRPULhx = Yes (%)	676.7 (18.9)	383.3 (17.9)		0.024
## RENALhx = Yes (%)	161.9 (4.5)	103.4 (4.8)		0.015
## LIVERhx = Yes (%)	255.9 (7.1)	151.8 (7.1)		0.001
## GIBLEDhx = Yes (%)	116.6 (3.3)	70.1 (3.3)		0.002
## MALIGHx = Yes (%)	861.0 (24.0)	523.2 (24.5)		0.011
## IMMUNhx = Yes (%)	944.2 (26.3)	571.5 (26.7)		0.009
## TRANShx = Yes (%)	418.9 (11.7)	255.7 (12.0)		0.009
## AMIhx = Yes (%)	120.6 (3.4)	68.5 (3.2)		0.009
## Cancer (%)				0.005
## No	2695.6 (75.2)	1604.4 (75.0)		
## Yes	618.8 (17.3)	369.5 (17.3)		
## Metastatic	271.6 (7.6)	164.5 (7.7)		
## Category (%)				0.055
## ARF	1505.3 (42.0)	938.4 (43.9)		
## COPD	286.5 (8.0)	160.5 (7.5)		
## CHF	299.0 (8.3)	174.8 (8.2)		
## Cirrhosis	146.2 (4.1)	79.9 (3.7)		

##	Coma	271.1 (7.6)	147.4 (6.9)	
##	Colon Cancer	4.4 (0.1)	3.0 (0.1)	
##	Lung Cancer	24.0 (0.7)	9.8 (0.5)	
##	MOSF w/Malignancy	281.2 (7.8)	161.2 (7.5)	
##	MOSF w/Sepsis	768.3 (21.4)	463.5 (21.7)	
##	Severity (mean (SD))	20.49 (5.31)	20.57 (5.24)	0.014

Covariate Balance Before and After Weighting

```
plot_cov_balance(
  data.set = data_set,
  covariates.added = covariates_added,
  weights.list = list(iptw = iptw, combined = combined_weights),
  weights.titles = c("IPTW", "Combined Weights"),
  cols = colors[2:4]
)
```



Case-Base Model Setup

```
## Set the case-base sampling ratio used across all case-base model fits.  
## A ratio of 100 samples 100 background person-moments per observed case.  
fsh_ratio <- 100
```

Counter-factual Survival Curve Setup

```
## Define the two treatment levels used to generate Counter-factual survival  
## curves: A = 0 represents no RHC and A = 1 represents RHC  
treatment_types <- c(0, 1)  
  
## Create a prediction dataset containing both treatment levels for estimation  
## of Counter-factual survival under each treatment condition  
survival_curves_data <- data.frame(A = treatment_types)  
  
# Time Grid Definition  
  
## Determine the maximum observed follow-up time in the analysis dataset  
tau_max <- max(data_set$time, na.rm = TRUE)  
  
## Restrict the primary survival-curve display to the first 150 days  
## (approximately four months) to focus on the relevant follow-up period  
focus_time <- 150  
  
## Define the time grid used for RMST integration, spanning the full observed  
## follow-up period at one-day intervals  
tau_grid <- seq(0, tau_max, by = 1)  
  
## Define the time grid used for plotting and estimating survival curves over  
## a restricted four-month follow-up period  
focused_grid <- seq(0, focus_time, by = 1)
```

Model 1: Unadjusted Marginal Structural Cox Model

```
unadjusted_cox <- fit_model(  
  fitting.function = survival::coxph,  
  formula = Surv(time, Delta) ~ A,  
  data.set = data_set,  
  seed = seed,  
  extract.print.hr = extract_print_hr,  
  se.func = vcov,  
  robust = TRUE  
)
```

```
## HR-hat: 1.164139  
## SE of HR-hat: 0.0342025  
## 95% CI: ( 1.088657 1.244854 )
```


Model 2: Unadjusted Case-Base Model

```
unadjusted_casebase <- fit_model(  
  fitting.function = FSH,  
  formula = Delta ~ A,  
  data.set = data_set,  
  seed = seed,  
  extract.print.hr = extract_print_hr,  
  se.func = sandwich::vcovHC,  
  ratio = fsh_ratio  
)
```

```
## HR-hat: 1.205197  
## SE of HR-hat: 0.03364396  
## 95% CI: ( 1.128288 1.28735 )
```

Model 3: Covariate-Adjusted Cox Model

```
covariate_adjusted_cox <- fit_model(  
  fitting.function = survival::coxph,  
  formula = as.formula(  
    paste("Surv(time, Delta) ~ A +", covariates_added)  
  ),  
  data.set = data_set,  
  seed = seed,  
  extract.print.hr = extract_print_hr,  
  se.func = vcov,  
  robust = TRUE  
)
```

```
## HR-hat: 1.288328  
## SE of HR-hat: 0.03765539  
## 95% CI: ( 1.196667 1.387009 )
```

Model 4: Covariate-Adjusted Case-Base Model

```
covariate_adjusted_casebase <- fit_model(  
  fitting.function = FSH,  
  formula = as.formula(paste("Delta ~ A +", covariates_added)),  
  data.set = data_set,  
  seed = seed,  
  extract.print.hr = extract_print_hr,  
  se.func = sandwich::vcovHC,  
  ratio = fsh_ratio  
)
```

```
## HR-hat: 1.371035  
## SE of HR-hat: 0.036884  
## 95% CI: ( 1.275417 1.473821 )
```

Model 5: Marginal Structural Cox Model (Intention-To-Treat)

```
ms_cox_itt <- fit_model(  
  fitting.function = survival::coxph,  
  formula = Surv(time, Delta) ~ A,  
  data.set = data_set,  
  seed = seed,  
  extract.print.hr = extract_print_hr,  
  se.func = vcov,  
  weights = iptw,  
  robust = TRUE  
)
```

```
## HR-hat: 1.215907  
## SE of HR-hat: 0.03794127  
## 95% CI: ( 1.128767 1.309775 )
```

Model 5: Marginal Structural Cox Model (Per-Protocol)

```
ms_cox_pp <- fit_model(  
  fitting.function = survival::coxph,  
  formula = Surv(time, Delta) ~ A,  
  data.set = data_set,  
  seed = seed,  
  extract.print.hr = extract_print_hr,  
  se.func = vcov,  
  weights = combined_weights,  
  robust = TRUE  
)
```

```
## HR-hat: 1.100938  
## SE of HR-hat: 0.04343661  
## 95% CI: ( 1.011088 1.198773 )
```

Model 6: IPW Case-Base Proportional Hazards Model (Intention-To-Treat)

```
ipw_casebase_itt <- fit_model(  
  fitting.function = FSH,  
  formula = Delta ~ A,  
  data.set = data_set,  
  seed = seed,  
  extract.print.hr = extract_print_hr,  
  se.func = sandwich::vcovHC,  
  weights = iptw,  
  ratio = fsh_ratio  
)
```

```
## HR-hat: 1.259992  
## SE of HR-hat: 0.03748826  
## 95% CI: ( 1.170731 1.356058 )
```

Model 6: IPW Case-Base Proportional Hazards Model (Per-Protocol)

```
ipw_casebase_pp <- fit_model(  
  fitting.function = FSH,  
  formula = Delta ~ A,  
  data.set = data_set,  
  seed = seed,  
  extract.print.hr = extract_print_hr,  
  se.func = sandwich::vcovHC,  
  weights = combined_weights,  
  ratio = fsh_ratio  
)
```

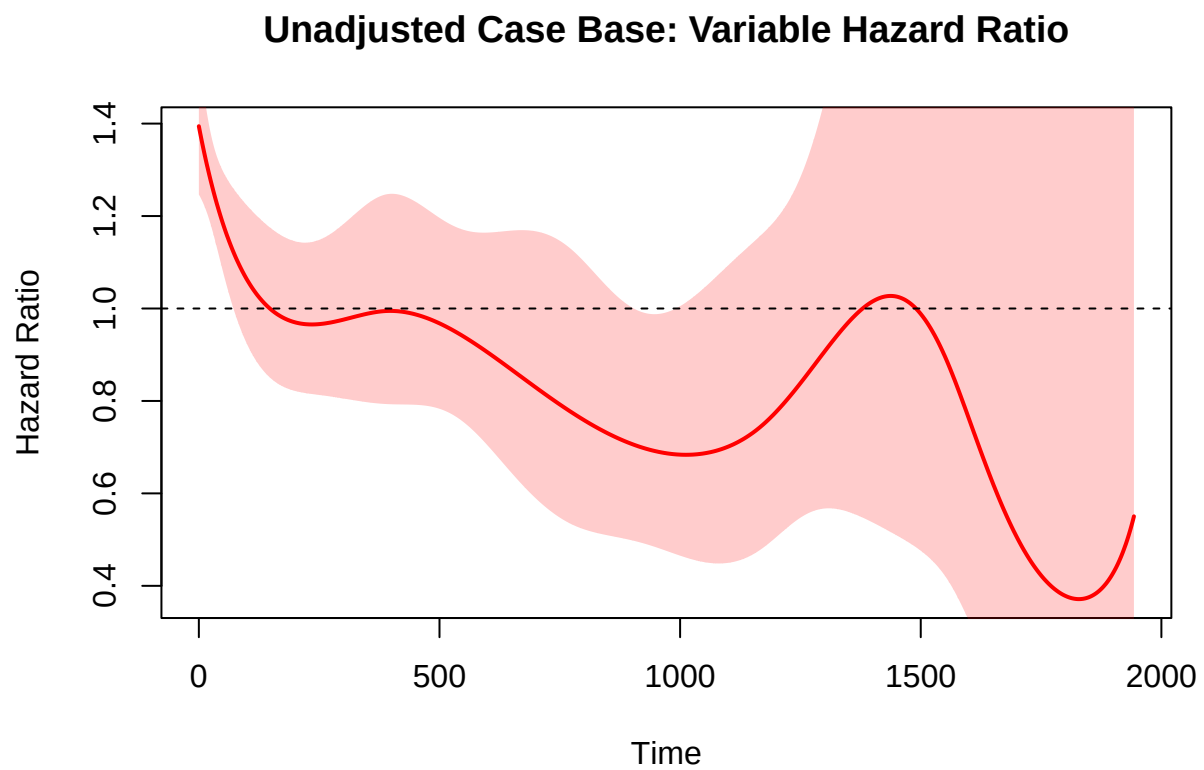
```
## HR-hat: 1.141811  
## SE of HR-hat: 0.03941902  
## 95% CI: ( 1.056915 1.233526 )
```

Model 7: Unadjusted Case-Base Model with Time-Varying Hazard Ratio

```
unadjusted_var_hr_casebase <- fit_model(  
  fitting.function = FSH,  
  formula = Delta ~ survival::pspline(time, df = 2) * A,  
  data.set = data_set,  
  seed = seed,  
  extract.print.hr = extract_print_hr,  
  constant.HR = FALSE,  
  ratio = fsh_ratio  
)
```

Time-Varying Hazard Ratio

```
plot_time_dep_hr(  
  list(unadjusted_var_hr_casebase),  
  main.title = "Unadjusted Case Base: Variable Hazard Ratio",  
  time.grid = tau_grid,  
  casebase.var.hr = casebase_var_hr  
)
```



Model 8: IPW Case-Base Model with Time-Varying Hazard Ratio (Intention-To-Treat)

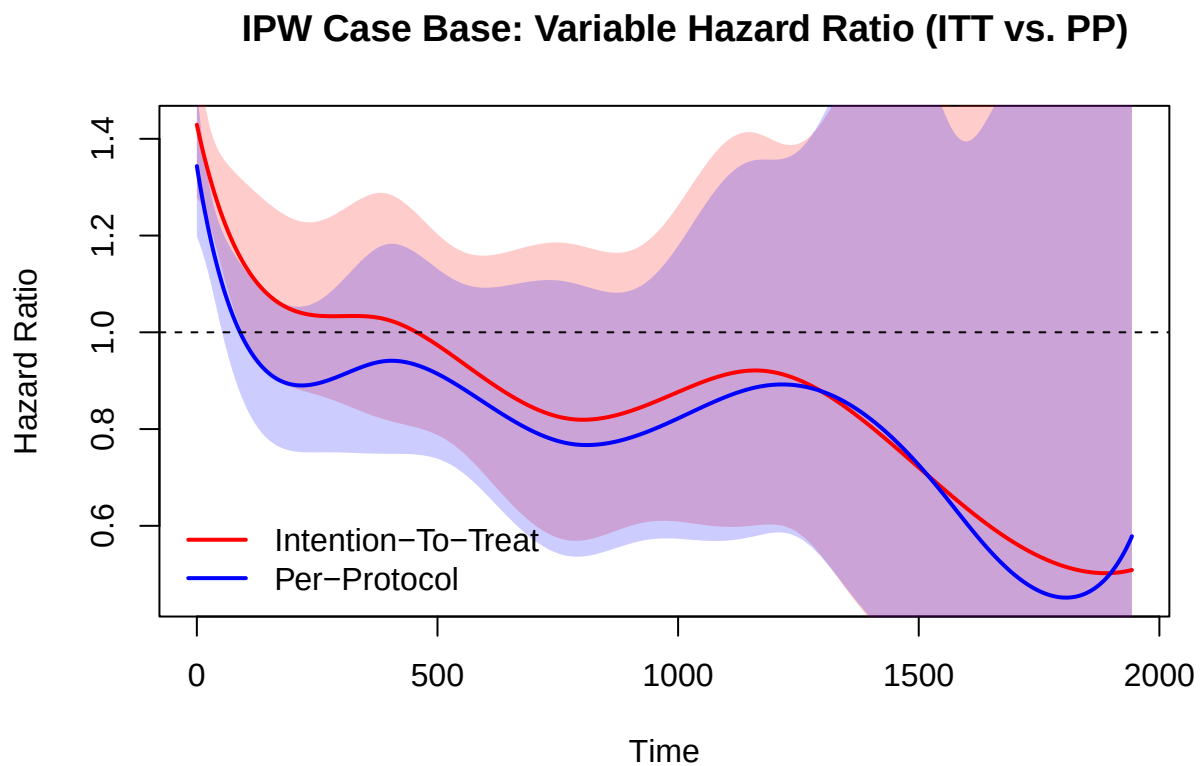
```
ipw_var_hr_casebase_itt <- fit_model(  
  fitting.function = FSH,  
  formula = Delta ~ survival::pspline(time, df = 2) * A,  
  data.set = data_set,  
  seed = seed,  
  extract.print.hr = extract_print_hr,  
  constant.HR = FALSE,  
  weights = iptw,  
  ratio = fsh_ratio  
)
```

Model 8: IPW Case-Base Model with Time-Varying Hazard Ratio (Per-Protocol)

```
ipw_var_hr_casebase_pp <- fit_model(  
  fitting.function = FSH,  
  formula = Delta ~ survival::pspline(time, df = 2) * A,  
  data.set = data_set,  
  seed = seed,  
  extract.print.hr = extract_print_hr,  
  constant.HR = FALSE,  
  weights = combined_weights,  
  ratio = fsh_ratio  
)
```

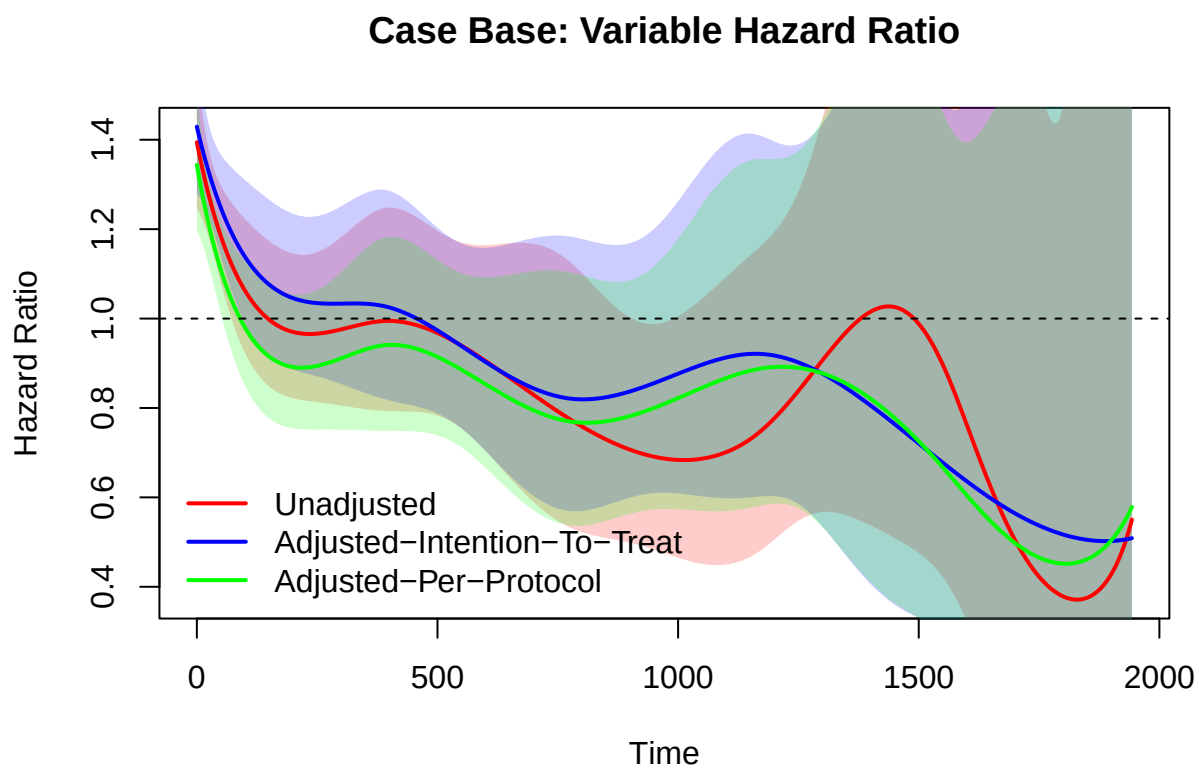

Time-Varying Hazard Ratio

```
plot_time_dep_hr(  
  list(ipw_var_hr_casebase_itt, ipw_var_hr_casebase_pp),  
  main.title = "  
    IPW Case Base: Variable Hazard Ratio (ITT vs. PP)  
  ",  
  time.grid = tau_grid,  
  casebase.var.hr = casebase_var_hr,  
  titles = c("Intention-To-Treat", "Per-Protocol"),  
  cols = colors[2:3]  
)
```



Head-to-Head Comparison: Models 7 and 8

```
plot_time_dep_hr(  
  list(  
    unadjusted_var_hr_casebase, ipw_var_hr_casebase_itt, ipw_var_hr_casebase_pp  
  ),  
  main.title = "Case Base: Variable Hazard Ratio",  
  time.grid = tau_grid,  
  casebase.var.hr = casebase_var_hr,  
  titles = c("Unadjusted", "Adjusted-Intention-To-Treat", "Adjusted-Per-Protocol"),  
  cols = colors[2:4]  
)
```



MS-Cox Counter-factual Survival Curves

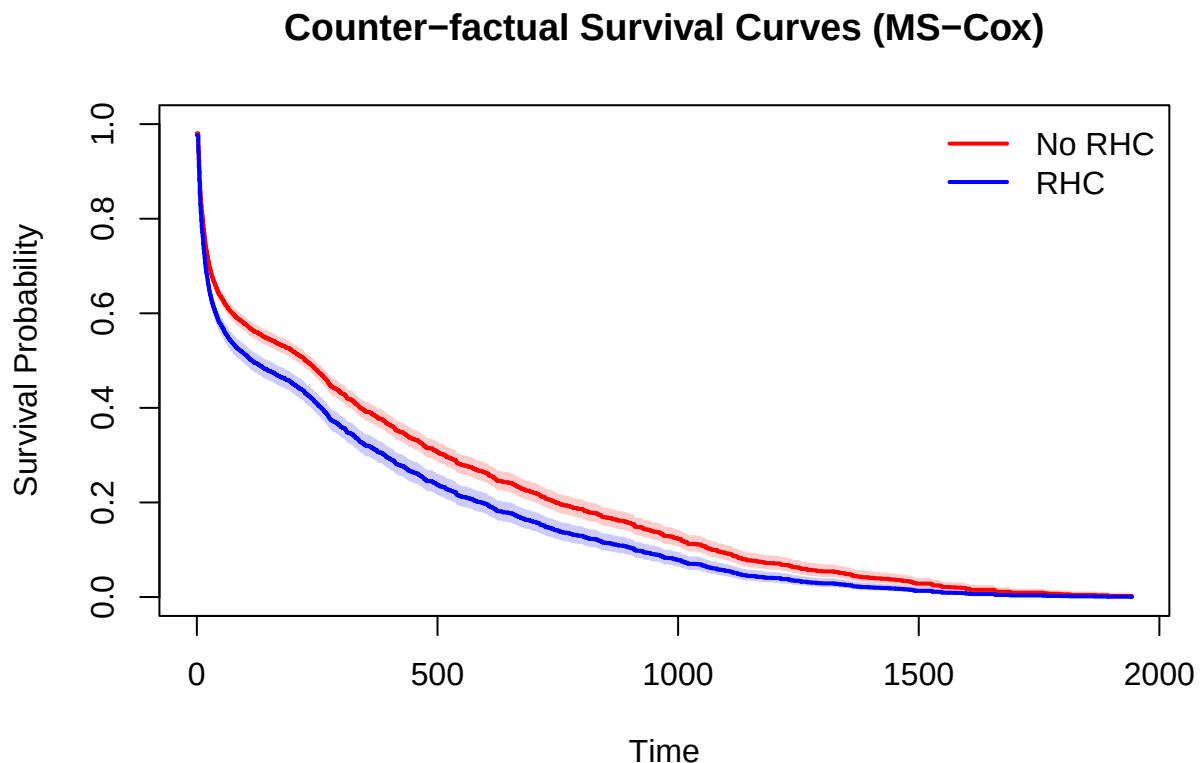
```
cox_csc_itt <- cox_csc(  
  model = ms_cox_itt,  
  new.data = survival_curves_data,  
  time.grid = tau_grid  
)  
  
cox_csc_pp <- cox_csc(  
  model = ms_cox_pp,  
  new.data = survival_curves_data,  
  time.grid = tau_grid  
)
```

IPW Case-Base Counter-factual Survival Curves

```
casebase_csc_itt <- casebase_csc(  
  seed = seed,  
  data.set = data_set,  
  model = ipw_var_hr_casebase_itt,  
  weights = iptw,  
  new.data = survival_curves_data,  
  time.grid = tau_grid,  
  FSH = FSH  
)  
  
casebase_csc_pp <- casebase_csc(  
  seed = seed,  
  data.set = data_set,  
  model = ipw_var_hr_casebase_pp,  
  weights = combined_weights,  
  new.data = survival_curves_data,  
  time.grid = tau_grid,  
  FSH = FSH  
)
```

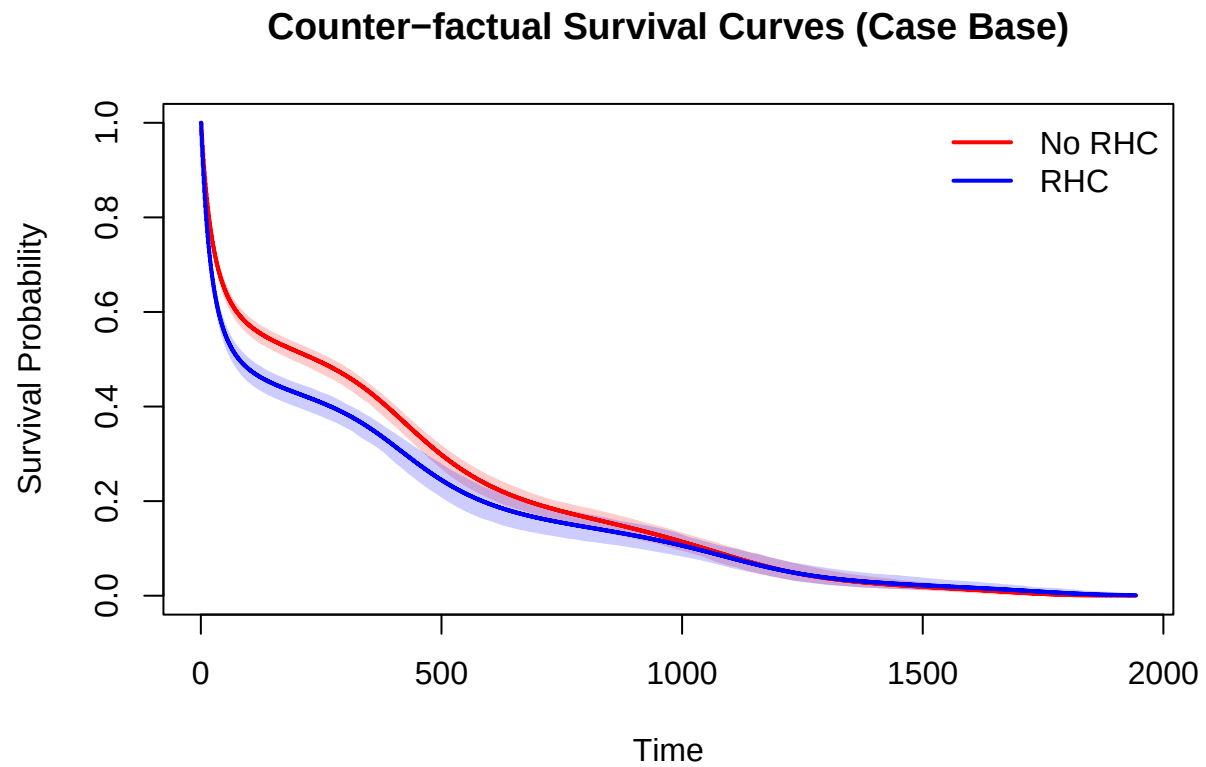
Head-to-Head Comparison of Counter-factual Survival Curves (Intention-To-Treat)

```
# Counter-factual Survival Curves (MS-Cox)
plot_csc(
  time.grid = tau_grid,
  S0 = cox_csc_itt$S0,
  S0.lower = cox_csc_itt$S0.lower,
  S0.upper = cox_csc_itt$S0.upper,
  S1 = cox_csc_itt$S1,
  S1.lower = cox_csc_itt$S1.lower,
  S1.upper = cox_csc_itt$S1.upper,
  main.title = "Counter-factual Survival Curves (MS-Cox)",
  fills = fills[2:3], cols = colors[2:3]
)
```



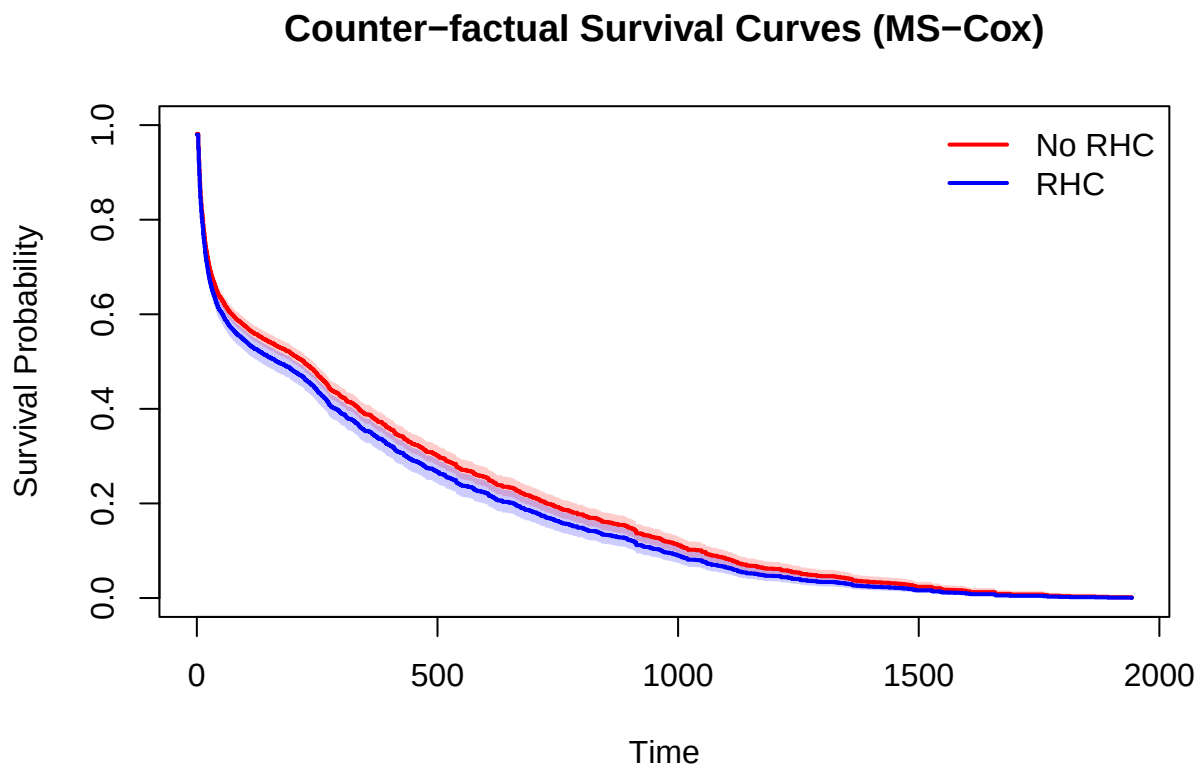
```
# Counter-factual Survival Curves (Case Base)
plot_csc(
  time.grid = tau_grid,
  S0 = casebase_csc_itt$S0,
  S0.lower = casebase_csc_itt$S0.lower,
  S0.upper = casebase_csc_itt$S0.upper,
  S1 = casebase_csc_itt$S1,
  S1.lower = casebase_csc_itt$S1.lower,
```

```
S1.upper = casebase_csc_itt$S1.upper,  
main.title = "Counter-factual Survival Curves (Case Base)",  
fills = fills[2:3], cols = colors[2:3]  
)
```



Head-to-Head Comparison of Counter-factual Survival Curves (Per-Protocol)

```
# Counter-factual Survival Curves (MS-Cox)
plot_csc(
  time.grid = tau_grid,
  S0 = cox_csc_pp$S0,
  S0.lower = cox_csc_pp$S0.lower,
  S0.upper = cox_csc_pp$S0.upper,
  S1 = cox_csc_pp$S1,
  S1.lower = cox_csc_pp$S1.lower,
  S1.upper = cox_csc_pp$S1.upper,
  main.title = "Counter-factual Survival Curves (MS-Cox)",
  fills = fills[2:3], cols = colors[2:3]
)
```



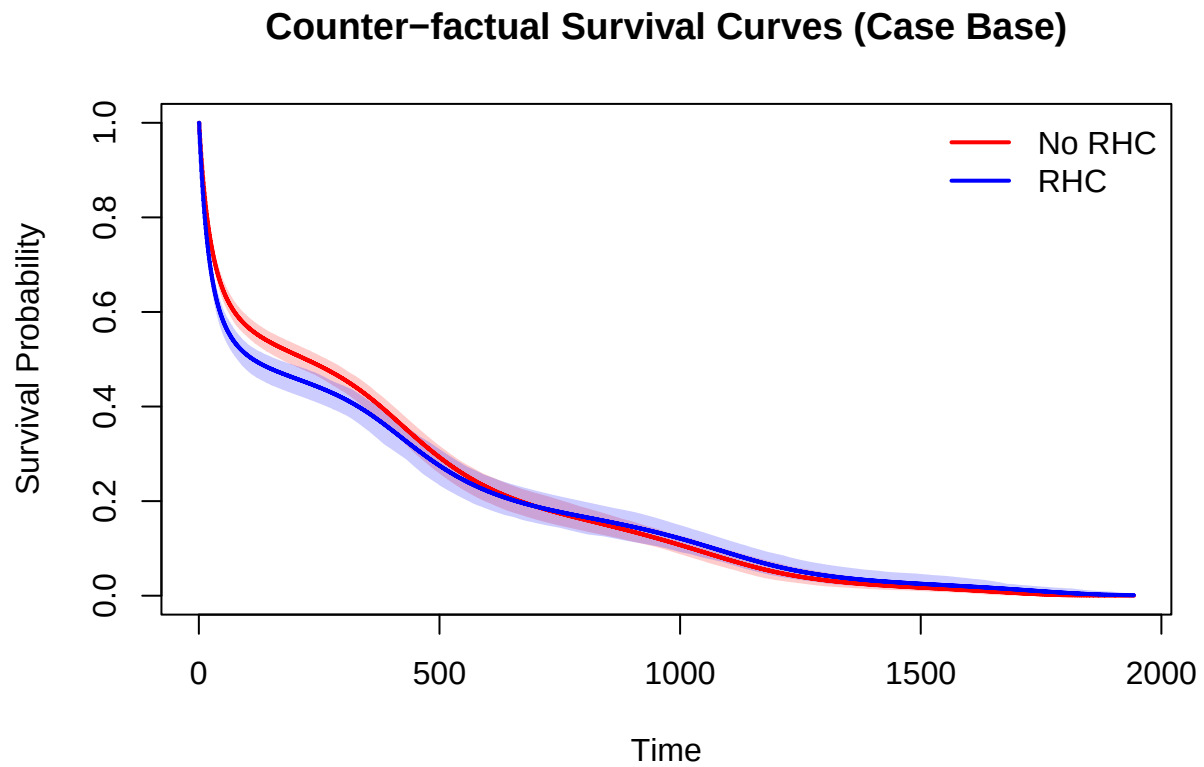
```
# Counter-factual Survival Curves (Case Base)
plot_csc(
  time.grid = tau_grid,
  S0 = casebase_csc_pp$S0,
  S0.lower = casebase_csc_pp$S0.lower,
  S0.upper = casebase_csc_pp$S0.upper,
  S1 = casebase_csc_pp$S1,
  S1.lower = casebase_csc_pp$S1.lower,

```

```

S1.upper = casebase_csc_pp$S1.upper,
main.title = "Counter-factual Survival Curves (Case Base)",
fills = fills[2:3], cols = colors[2:3]
)

```

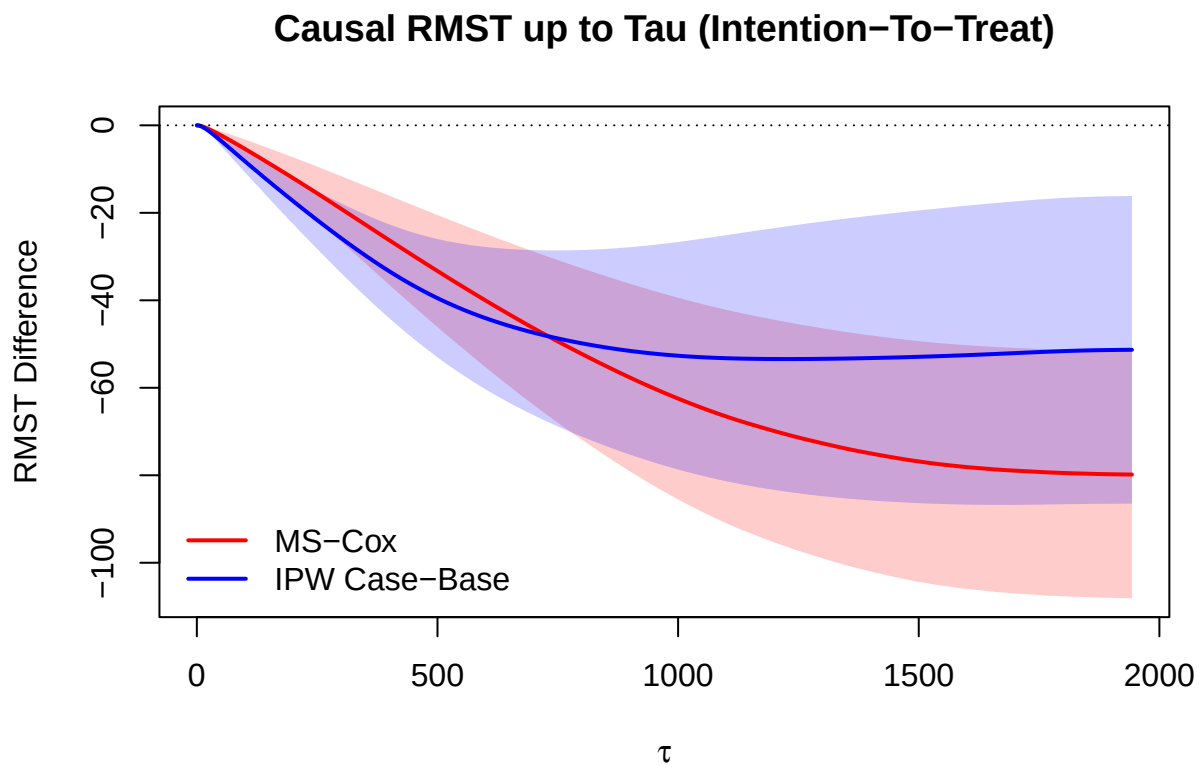


RMST Estimates and Standard Errors

```
rmst_stats_itt <- rmst_stats(  
  models.list = list(  
    cox = ms_cox_itt,  
    casebase = ipw_var_hr_casebase_itt  
  ),  
  time.grid = tau_grid,  
  new.data = survival_curves_data,  
  cox.rmst.list = cox_rmst_list,  
  casebase.rmst.list = casebase_rmst_list,  
  rmst.calculation = rmst_calculation,  
  rmst.diff = rmst_diff,  
  casebase.surv = casebase_surv  
)  
  
rmst_stats_pp <- rmst_stats(  
  models.list = list(  
    cox = ms_cox_pp,  
    casebase = ipw_var_hr_casebase_pp  
  ),  
  time.grid = tau_grid,  
  new.data = survival_curves_data,  
  cox.rmst.list = cox_rmst_list,  
  casebase.rmst.list = casebase_rmst_list,  
  rmst.calculation = rmst_calculation,  
  rmst.diff = rmst_diff,  
  casebase.surv = casebase_surv  
)
```


RMST Comparison Plots

```
plot_rmst_comparison(  
  rmst.stats = rmst_stats_itt,  
  fills = fills[2:3], cols = colors[2:3],  
  additional.title = "(Intention-To-Treat)"  
)
```



```
plot_rmst_comparison(  
  rmst.stats = rmst_stats_pp,  
  fills = fills[2:3], cols = colors[2:3],  
  additional.title = "(Per-Protocol)"  
)
```

Causal RMST up to Tau (Per-Protocol)

